

Epiq Revenue-Leakage — Team Briefing

EVEglyphDesign · Anchored to Tobias Polly, polly consulting e.U. (17 July 2026) · Status: Propose-only

Where to go

Onboarding surface: eveglyphdesign.github.io/epiq-revenue-leakage
Repository: github.com/EVEglyphDesign/epiq-revenue-leakage
Open the surface, get connected, spin the model. Everything works off one repo.

What this is

A fork off of EVE — the EVE-side baseline. We use it to clear up the requirements and stand up the initial demo. That is the whole job right now.

- All Epiq data stays inside the Epiq environment. Demo data lives in Postgres, inside Epiq infrastructure. None of Epiq's data is handled outside it.
- No proprietary Epiq information in this repo. The client-confidential side is set up and governed by Tobias, walled off — we contribute through him.
- On approval, the demo becomes a repository build on Epiq infrastructure. The EVE fork copies back into the Epiq environment and we continue there.
- We do not own the capital asset — that is Epiq's.

The model in one line

Revenue leakage in enterprise services is correctable — but only if you catch it in the cycle it happens. This is a reasoning agent grounded in the client's own edit history, not another dashboard or narrative.

The pattern-analysis pipeline — who owns each stage

Order matters. It starts with Matt handing off where the SAP data sits, Satya readying and reconciling it, and Seshu piping it out via DataSphere — that live feed is what the already-running engine reads from, before a validation gate opens it in Teams. Full architecture in Document B.

#	Stage	What happens	Owner	Status
1	Locate the SAP data (draft-billing)	Surface where all the SAP data sits behind the draft-billing process Matt has stewarded since Dany's departure — the source of everything downstream	Matt Kephart	Hand off — first
2	Ready & reconcile the data	Get that SAP data ready and reconcile it so it is fit to pipeline — must happen before the live connection	Satya	Prep

pour le bien-être du peuple

#	Stage	What happens	Owner	Status
3	Live SAP → DataSphere pipeline	Stand up the live connection and pipeline the reconciled SAP data out via DataSphere — the feed everything else reads from	Seshu (with Satya)	Build
4	Source adapter	Crosses the boundary into Epiq's live system; presents edit history in one uniform shape	Tobias (points it)	Configure
5	Reasoning agent	Bounded Router → Executor → Synthesizer; plans, fetches, narrates in plain language	Tobias — reused unchanged	Live
6	Compute layer	Ephemeral DuckDB (+ graph if needed); spun up per session, torn down after	Tobias	Configure
7	Model deployment	Azure AI Foundry model confirmed in-tenant, inside the network boundary	Tobias	Configure
8	Delivery (Teams)	Users ask in plain English, get answer + card; no SQL, no dashboard	Tobias — reused unchanged	Live
9	Observability	Langfuse records every step end-to-end; any answer replayable	Tobias — reused unchanged	Live
10	Revenue validation gate	Validate leakage answers against a known period — first target: the draft-billing auto-billing patterns — then open in Teams	Tobias validates · Dany/PMO reviews	Pending

What Tobias owns to complete — the short list

The data has to flow first: Matt hands off where the SAP data sits, Satya readies and reconciles it, Seshu pipes it out via DataSphere. Only then does Tobias's side read from it. Everything client-side and confidential is Tobias's, walled off from this repo. To finish the retargeting from architecture to a live Revenue Leakage demo:

1. Point the source adapter at the live SAP → DataSphere edit-history feed once Matt, Satya, and Seshu have the reconciled data flowing.
2. Stand up the ephemeral DuckDB layer (add the graph only if relationship questions demand it).
3. Confirm the model deployment and network boundary in Epiq's tenant.
4. Supply the revenue-specific model detail the architecture extract does not yet carry (the gap — cued in Document B): exact leakage measures / field rules (which post-booking edits count, thresholds); the finding schema the Synthesizer emits for revenue answers; the known-period validation set used to prove the answers.
5. Validate against a known period — first target the draft-billing auto-billing patterns — then open it in Teams.

Steps 1–3 and 5 are configuration on the running engine. Step 4 is the only net-new input, and it comes from Tobias — nothing in this repo invents it.

What Matt owns — the draft-billing data (this comes first)

Five years ago we improved revenue performance with a draft-billing process, implemented with Matt Kephart; Matt has been its steward since Dany's departure. Nothing about the working process changes — we do not touch the UI, and we do not touch anything already working.

Matt's contribution is the data: surface where all the SAP data behind the draft-billing process sits (table names, keys, relationships, schema exports/DDL) and hand it off. This is the source of everything downstream — it has to happen before Satya can ready and reconcile it, and before Seshu can pipeline it.

The leakage hypothesis lives here. We think there is probably leakage in the auto-billing, and that we can improve it — but we need to run the pattern analysis on it to confirm before we act. We are twinning the data side only, so the engine can read the auto-billing patterns without disturbing the live process.

What Satya does — ready and reconcile

Once Matt has handed off the SAP data, Satya gets it ready and reconciles it so it is fit to pipeline. This is the prep step that has to land before the live connection is stood up.

What Seshu does — the live pipeline

With the reconciled data in hand, Seshu (working with Satya) stands up the live SAP → DataSphere connection and pipelines it out via DataSphere. That live feed is what the engine reads from.

Getting connected (short version)

1. Free GitHub account (if you don't have one).
2. Send your GitHub username to the group to be added.
3. Clone the repo (or press . on the repo page for the browser editor).
4. Point your surface of choice at the folder (Cursor, VS Code + Copilot, Claude/ChatGPT, or the GitHub web editor).
5. Make changes → commit → push. That flows back to the shared model and the surface.

pour le bien-être du peuple